

Tag — A Multi-Agent Reinforcement Learning Laboratory

Group: Reinforcement Forgetting (Track A)

April 26, 2026

1. Problem Statement

This project investigates how different reinforcement learning algorithms learn asymmetric pursuit–evasion behavior in a multi-agent **Tag** environment. We implement a 2D tile-based game in which one **tagger** chases multiple **runners** across a map with walls and movable obstacles, and train agents to play both roles through self-play.

Tag is a minimal but information-rich setting where emergent strategies — pursuit, evasion, cornering, and obstacle use — are directly observable. The role asymmetry further lets us study how each algorithm handles role-specific credit assignment within a single shared training loop.

The **research gap** we address is that most multi-agent RL benchmarks fix each agent’s role for the entire episode, while Tag *transfers* the tagger role on every successful tag. This requires the same agent to maintain two opposite policies and switch between them mid-episode. We provide a unified benchmark that compares tabular, deep value-based, and deep policy-gradient methods on this role-switching task using a common environment, observation, reward function, and evaluation protocol.

2. Related Work

- **Tabular methods** — Q-Learning and SARSA form our baselines; their off-policy vs. on-policy update rules motivate our role-specific analysis (e.g., risk-seeking taggers vs. risk-averse runners).
- **Deep value-based RL** — DQN uses a neural Q-network with experience replay and a target network for stability.
- **Deep policy-gradient RL** — PPO uses a clipped surrogate objective and is the standard for multi-agent simulation environments.
- **Multi-agent pursuit-evasion** — OpenAI’s Hide-and-Seek demonstrated emergent tool-use in a similar setting using PPO with self-play, but with fixed roles per episode.

Difference from prior work. We compare four algorithms across three method families on the *same* environment with role-switching dynamics, and we evaluate each role *independently* (trained role vs. random opponent) — a protocol that is rare in the existing multi-agent RL literature.

3. Methodology

3.1 Environment

A 2D grid-based Tag game built on Pygame with walls, spawn points, and movable crates. The default map hosts six agents (one tagger, five runners) at any time, with tag events transferring the tagger role. Parallel headless simulations support fast CPU-based training; an optional Pygame display mode supports live behavioral inspection.

3.2 Observation (ego-centric)

Each agent observes a normalized, self-centered view: own position and velocity; relative position and distance of every other active agent (with the tagger and nearest runner highlighted); 8-direction wall raycasts (normalized distance to the nearest wall along each ray); relative positions of nearby movable crates. Eliminated agents are filtered out so policies do not perceive inactive opponents.

3.3 Reward Design (Pure-Proximity, “Plan C”)

After iterating through several earlier designs, we converged on a minimal, interpretable scheme: terminal signals dominate (+50 on tag, −80 on being tagged, with the asymmetry pushing runners toward caution and taggers toward aggression); a **quadratic proximity field** within a fixed 140-pixel radius gives the tagger a bonus and the runner a symmetric penalty, with the steepest gradient near the tag zone; constant time pressure (−0.05/step for the tagger) and constant survival bonus (+0.10/step for the runner), with no time-scale amplifier. Long-range navigation is delegated to the observation; the reward is silent outside the proximity radius, forcing the policy to rely on perception rather than reward gradients for coarse movement.

3.4 Dual-Role Architecture

Agents share **two** networks — one *tagger brain* and one *runner brain* — selected at runtime based on the agent’s current role. All six agents contribute experience to the same shared networks, dramatically improving sample efficiency while preserving role-specific specialization.

3.5 Training and Evaluation Pipeline

A unified `run_experiment.py` pipeline handles the full workflow for any algorithm: (1) train with periodic checkpoints at configurable epoch milestones plus a *best-so-far* checkpoint that captures the true policy peak — important for algorithms such as PPO that oscillate round-to-round; (2) joint evaluation of each checkpoint, with the policy frozen at the algorithm level (preventing stale transitions from polluting the network during eval) and a fixed random seed (ensuring fair cross-checkpoint comparison); (3) isolated role evaluation; (4) plotting of training curves, per-epoch performance bars, role-trend lines, role diagnostics, trajectories, and position heatmaps.

4. Evaluation Plan

Metrics. (i) Joint mean tags per episode at each checkpoint (overall policy quality with both roles trained), accompanied by **rendered evaluation videos** so we can visually recognize the moving patterns of tagger and runner (e.g., direct pursuit vs. wall-hugging escape, line-of-sight prediction, cornering); (ii) tagger-isolated tags — trained tagger vs. untrained runners (more tags = better tagger); (iii) runner-isolated tags — trained runners vs. untrained tagger (fewer tags = better runners); (iv) mean steps between tags (proxy for episode-level pursuit/evasion speed); (v) behavioral diagnostics — trajectory plots and 2D position heatmaps to identify pathological patterns such as corner-camping, standing still, and reward-farming.

Experimental settings. Four algorithms (Q-Learning, SARSA, DQN, PPO) on the same map, observation, reward function, and decision interval. Each algorithm is trained for **100 epochs**, with checkpoints saved at epochs {2, 5, 10, 50, 100} plus a *best-so-far* checkpoint. 20 evaluation episodes × 1000 steps per checkpoint, fixed seed across algorithms for fair comparison. Two parallel headless simulations during training pool experience into the shared networks.

Expected results. A reproducible RL pipeline supporting headless CPU training and optional live visualization; trained checkpoints and evaluation metrics for each algorithm at every saved epoch; a comparative report analyzing algorithm performance, role-specific behavior, and reward sensitivity; trajectory visualizations and diagnostic plots for qualitative behavioral analysis.

Team & timeline. Each team member owns one algorithm’s implementation and experiments while sharing the common environment, pipeline, and evaluation protocol so results are directly comparable. Tentative milestones: setup & environment (Weeks 1–2); tabular and DQN baselines (Weeks 3–4); PPO training and tuning (Weeks 5–6); cross-algorithm evaluation and behavioral analysis (Week 7); final report and presentation (Week 8).